Evidence-Grounded AI Agents for Browser-Based Web Security Assessment: A Controlled Playwright Framework with Structured Tool Outputs and CWE/CVE Mapping

Author Name

Department of Computer Science and Engineering, Institution Name, Bengaluru, India

corresponding.author@example.com

Draft manuscript for author review. Replace placeholder author details and verify all claims before submission.

Abstract

AI-assisted browser automation can make web security assessment more accessible, but unconstrained agents may repeat actions, misinterpret page state, and report plausible-sounding vulnerabilities without adequate evidence. This paper presents a controlled framework for browser-based security assessment in which a large language model proposes bounded actions while a deterministic Playwright tool layer executes them, returns schema-validated results, and captures reproducible evidence. The framework separates navigation and observation, candidate-check execution, evidence validation, weakness classification, severity assessment, and report generation. It introduces action budgets, state-aware loop detection, idempotency keys, typed tool responses, and explicit stop conditions to reduce repeated actions and malformed outputs. Security findings are grounded in observable artifacts such as response headers, DOM state, browser-console events, and controlled test outcomes. Common Weakness Enumeration (CWE) identifiers are used to describe weakness classes, while Common Vulnerabilities and Exposures (CVE) identifiers are attached only when a specific known vulnerability is supported by product, version, and applicability evidence. The design is intended for applications owned by the operator or explicitly authorized for testing. A prototype workflow and initial local-application observations motivate the design; however, no benchmark-scale detection rates or comparative performance claims are made because a labelled evaluation dataset has not yet been collected. The paper defines a reproducible evaluation plan covering finding precision, recall, evidence completeness, action efficiency, and false-positive rate.

Keywords: AI agents · Web application security · Playwright · Evidence-based testing · CWE · CVE

1 Introduction

Modern web applications expose complex user journeys through forms, routes, asynchronous requests, client-side rendering, and third-party components. Security testing must account for both server responses and the browser-visible behaviour that emerges as users navigate these interfaces. Browser automation frameworks such as Playwright can observe rendered pages, interact with controls, inspect network responses, and preserve screenshots or traces. Large language models (LLMs) add a flexible planning layer: they can interpret page context, select candidate checks, and translate technical observations into readable explanations. Yet combining these capabilities does not automatically produce a reliable security scanner.

An agent may click the same button repeatedly, confuse a navigation control with a test action, emit malformed tool arguments, or treat an input field accepting a string as proof of cross-site scripting (XSS). It may also mistake a missing hardening header for a confirmed exploitable vulnerability. These errors are especially important in security work because reports influence remediation priorities and can trigger unnecessary engineering effort. A useful system must therefore make the execution path bounded, the tool interface typed, and every finding traceable to evidence.

This paper proposes a controlled architecture for an LLM-assisted browser security agent. The LLM is not given unrestricted control over the browser. Instead, it requests actions from a small tool layer that validates arguments, checks policy, executes one action at a time, and returns structured results. Deterministic checks identify observable conditions; an evidence validator determines whether the condition meets the minimum proof standard; and the LLM is used to explain findings rather than invent their underlying facts.

The work focuses on four contributions. First, it describes action-loop controls based on budgets, state fingerprints, repeated-action detection, and explicit termination rules. Second, it specifies structured tool outputs that separate execution status, page state, evidence, and errors. Third, it presents an evidence-based finding model that distinguishes observations, potential weaknesses, validated vulnerabilities, and inconclusive tests. Fourth, it defines a cautious mapping process for CWE and CVE identifiers, keeping weakness taxonomy separate from known-vulnerability identification and severity scoring.

The intended use is defensive assessment of applications owned by the operator or covered by explicit written permission. Active tests are limited to a defined target scope, and high-impact actions require policy approval. The framework is a design and prototype report rather than a claim that the agent already detects vulnerabilities at a known accuracy level.

2 Background and Related Work

2.1 Browser Automation for Security Testing

Browser automation enables repeatable interaction with rendered applications. Playwright provides APIs for navigation, locators, form interaction, browser contexts, and inspection of network and page events. Such observations are valuable for tests that depend on client-side routes, dynamically rendered controls, or the sequence of actions required to reach a state. Automation, however, only supplies mechanisms for observation and interaction. It does not by itself decide whether an observed condition is a security weakness or whether an exploitability claim is justified.

2.2 LLM-Based Tool Use

Tool-using LLM systems select actions from a set of available functions and reason over returned observations. This flexibility creates a boundary problem: free-form model text can be mistaken for a tool call, arguments can be malformed, and the model can repeat an action without checking whether the state changed. The framework therefore treats model output as a proposal, not as executable authority. A schema validator and policy gate sit between the model and the browser.

2.3 Security Testing Guidance and Weakness Taxonomies

The OWASP Web Security Testing Guide (WSTG) provides a structured methodology for testing web application security. The framework uses testing guidance as a source of candidate test categories, while keeping each check specific to the target and authorization scope. CWE provides a vocabulary for describing classes of software weakness. CVE identifies a publicly disclosed vulnerability instance; it is not a generic label for any security concern. CVSS provides a severity scoring system and should not be confused with either CWE or CVE. These distinctions prevent a common reporting error: assigning a CVE to a generic configuration issue without evidence that the target contains the affected product and vulnerable version.

2.4 Evidence and Reproducibility

A security finding should be independently reviewable. Useful evidence includes the target URL, timestamp, HTTP status, relevant response headers, a redacted DOM excerpt, console or network events, screenshots, and the exact check that produced the observation. Browser video is useful for reconstructing the agent's journey, but it does not prove a vulnerability by itself. Evidence should be stored with provenance and should not contain credentials, session tokens, or unnecessary personal data.

3 Problem Definition and Design Goals

The system accepts a target application, an authorization record, a test policy, and a set of permitted checks. It produces a report containing observations and findings, each linked to evidence and a confidence state. The main reliability problem is to prevent the planner's uncertainty from becoming an unverified security claim.

The design has six goals. (1) Bounded execution: every run has maximum step, time, navigation, and request budgets. (2) State-aware progress: repeated actions are stopped when the page state and action result indicate no meaningful change. (3) Typed interfaces: every tool returns a valid object conforming to a versioned schema. (4) Evidence-first findings: no vulnerability is reported as confirmed without a check-specific proof condition. (5) Taxonomy correctness: CWE describes the weakness class, CVE is used only for a supported known vulnerability, and severity is documented separately. (6) Authorization and minimization: the target allowlist, test depth, and prohibited actions are checked before execution.

The system is not intended to replace expert penetration testing, source-code review, or a full dynamic application security testing platform. It is a browser-oriented assistant for repeatable, bounded checks and structured reporting. It cannot infer server-side implementation details from browser evidence alone, and it must report such uncertainty explicitly.

4 Proposed Architecture

The architecture consists of six cooperating components.

4.1 Policy and Scope Gate

Before browser launch, the operator records the target origin or explicit list of origins, the permission basis, the allowed test categories, rate limits, and prohibited actions. The gate rejects navigation outside the allowlist unless an explicit approved redirect rule exists. It also prevents destructive operations, account creation, data modification, bulk requests, and authenticated testing unless the policy specifically permits them.

4.2 LLM Planner

The LLM receives a compact observation: current URL, page title, visible interactive elements, recent navigation events, and a list of available tools. It may request one action at a time, with a reason and expected state change. It cannot call arbitrary shell commands or invent browser APIs. A planner response that fails schema validation is rejected and counted against a small invalid-output budget.

4.3 Playwright Tool Adapter

The adapter exposes a narrow set of operations such as navigate, inspect page, click a uniquely identified locator, fill an approved test field, capture screenshot, and inspect response headers. Each operation validates input length, selector safety, target origin, timeout, and allowed action class. The adapter executes one action and returns the resulting URL, title, selected DOM observations, relevant events, elapsed time, and an explicit outcome.

4.4 Deterministic Check Engine

Checks operate on structured observations rather than on model-generated prose. Examples include presence and value of security headers, unexpected console errors, mixed-content observations, form attributes, and safe validation of expected behaviour in a deliberately vulnerable test fixture. Each check declares its preconditions, method, evidence requirements, and possible result states.

4.5 Evidence Validator and Classifier

The validator checks whether the evidence is complete and sufficient for the stated conclusion. It separates a raw observation (for example, a missing Content-Security-Policy header) from a weakness interpretation (a defense-in-depth gap) and from a confirmed vulnerability (a demonstrated security impact). The classifier maps supported weakness classes to CWE candidates and only adds a CVE when matching evidence supports the affected product, version, configuration, and vulnerability conditions.

4.6 Report Builder

The report builder produces machine-readable JSON and a human-readable report. Each finding includes title, affected URL, observation, expected condition, evidence references, confidence, impact, severity rationale, CWE mapping, optional CVE match, remediation, and limitations. LLM-generated prose is generated from this validated record and cannot overwrite the underlying evidence.

5 Action-Loop Handling and Structured Tool Outputs

5.1 Bounded State Machine

The agent operates as a finite-state controller: INITIALIZE, OBSERVE, PLAN, VALIDATE_ACTION, EXECUTE, VERIFY_STATE, CLASSIFY, and STOP. Every transition is logged. The agent stops when the step budget is exhausted, the same action-state pair repeats beyond a threshold, no allowed action remains, the page enters an unsupported state, or a safety rule is triggered. A practical starting policy is a modest per-run step limit and a small retry allowance; exact values should be tuned during evaluation rather than treated as universal constants.

5.2 State Fingerprints and Duplicate Detection

A state fingerprint can be computed from normalized URL, page title, a stable digest of visible interactive elements, and a small set of navigation/network markers. The action signature includes tool name, normalized locator or target, and a digest of the arguments. If the same action signature is issued against an equivalent state repeatedly without a meaningful state transition, the controller blocks it and asks the planner to observe or stop. Sensitive field values are excluded from fingerprints.

5.3 Idempotency and Post-Action Verification

Actions that can cause side effects should not be retried automatically. The adapter assigns a run-local action identifier and records whether execution started, completed, or became uncertain. After a click or navigation, the controller checks the actual URL, page state, and expected event. A click that times out after possibly reaching the server is treated as an uncertain outcome, not blindly repeated. Read-only observations may be retried under a limited retry policy.

5.4 Typed Tool Contract

Each tool response follows a versioned schema with fields for status, action identifier, timestamp, target URL, before/after state, evidence references, and error information. Tool output is never a mixture of JSON fragments, free-form commentary, and model channel markers. A simplified response is shown below:

```
{ "schema_version": "1.0", "action_id": "run-01-step-04", "status": "completed", "tool": "c
lick", "target_url": "http://127.0.0.1:5176/auth", "state_changed": true, "evidence": [{"ty
pe": "url", "value": "http://127.0.0.1:5176/auth"}], "error": null }
```

The actual implementation should validate every response against a JSON Schema or an equivalent runtime validator. Model explanations should be stored in a separate field from raw tool results. Invalid outputs should be rejected and logged without being interpreted as evidence.

5.5 Loop-Handling Algorithm

At each iteration, the controller observes the page, computes the state fingerprint, requests one proposed action, validates the action against policy, and checks whether the action-state signature has already occurred. If allowed, the tool executes once and returns a typed result. The controller verifies the post-action state and records whether progress occurred. A repeated signature, unchanged state after repeated attempts, invalid model response, or budget exhaustion causes a controlled stop or a request for a new observation. This prevents repeated login clicks and duplicated records from being interpreted as meaningful progress.

6 Evidence-Based Vulnerability Checks

Each check is implemented as a deterministic procedure with a defined proof standard. The LLM can suggest a check, but it cannot declare the check passed or failed independently of the returned evidence.

6.1 Security Header Inspection

The adapter records response status and selected headers for each in-scope response. A missing header is an observation, not automatically a critical vulnerability. For example, a missing Content-Security-Policy may indicate a defense-in-depth gap; its risk depends on the application and other controls. Missing Strict-Transport-Security matters in the context of HTTPS deployment and domain policy. X-Frame-Options and CSP frame-ancestors relate to framing protections. X-Content-Type-Options can reduce MIME-sniffing risks. Findings should include the response URL, header name, actual value or absence, and the route tested.

6.2 Input Validation and XSS Verification

Entering a string such as <script>alert(1)</script> into an email field is not evidence of XSS. The field may reject the value, store it as text, or never render it. A controlled XSS check requires a permitted test fixture or safe test account, a defined injection point, a subsequent rendering context, and observable evidence that untrusted input is interpreted as executable content. Tests should avoid exfiltration, destructive payloads, or interactions with real users. Where execution cannot be safely verified, the result must be “inconclusive” or “input accepted; exploitability not demonstrated,” not “XSS confirmed.”

6.3 Navigation and Route Checks

Route exploration should use a queue of discovered in-scope links and stable identifiers rather than repeated generic clicks. Each route is normalized and deduplicated. External links are recorded but not followed unless allowed. A navigation failure is not itself a security vulnerability; it may be a usability or testability issue.

6.4 Browser and Network Events

Console errors, failed requests, redirects, and mixed-content events may provide context but should not be overclassified. A JavaScript exception can be a bug without a security impact. The finding model should identify the event and explain why it is security-relevant, if at all. Request and response bodies should be captured only when necessary and should be redacted.

6.5 Check Result States

Every check returns one of: PASS (the tested property was observed as expected), FINDING (the proof threshold was met), INCONCLUSIVE (required evidence unavailable or ambiguous), NOT_APPLICABLE (preconditions not met), or ERROR (the check could not run). This vocabulary prevents the common mistake of treating missing evidence as proof of safety or treating tool failure as proof of vulnerability.

7 CWE/CVE Mapping and Severity

CWE and CVE play different roles in the report. CWE identifies a class of weakness, such as improper input neutralization or missing security configuration. CVE identifies a particular publicly disclosed vulnerability. CVSS communicates severity using a defined scoring framework. The report must not substitute one identifier type for another.

7.1 CWE Candidate Mapping

A finding is mapped to a CWE only when the observed behaviour supports that weakness class. For example, a demonstrated injection flaw may be mapped to an appropriate injection-related CWE after confirming the output context. A missing security header may map to a relevant security-configuration weakness when the mapping is applicable, but the report should not force a CWE if the evidence only supports a hardening observation. The mapping record should include the candidate CWE ID, its official title, rationale, and confidence.

7.2 CVE Matching Rules

A CVE match requires evidence beyond a generic symptom. The system must identify a product or component, obtain a credible version signal, compare it against the affected-version range and prerequisites in a trusted vulnerability source, and document why the advisory applies. If the browser-visible evidence cannot establish product or version, the system should leave the CVE field empty and say “no CVE assigned; insufficient product/version evidence.” It must never generate a CVE identifier from the LLM’s memory alone. A CVE lookup can provide candidate records, but the match must be reviewed against authoritative advisory data and target evidence.

7.3 Severity and Confidence

Severity should be based on demonstrated impact, exploit prerequisites, affected scope, and the availability of compensating controls. CVSS may be used where the scoring inputs are justified; a guessed numerical score should not be presented as objective. Confidence is a separate dimension: a high-impact hypothesis can have low confidence if the evidence is weak. The report should therefore include both severity and confidence, with an explanation for each.

7.4 Example Finding Record

A header observation could be represented as: category “security configuration”; observation “Content-Security-Policy was absent on the tested HTML response”; evidence “response headers for the in-scope route”; status “hardening recommendation”; CWE “candidate only if the chosen weakness definition fits”; CVE “none”; severity “context-dependent”; confidence “high that the header was absent, low-to-medium that the absence alone creates exploitable impact.” This formulation is more accurate than asserting a CVE or critical vulnerability from the header observation alone.

8 Authorization, Safety, and Data Governance

The system must run only against a target owned by the operator or explicitly authorized in writing. The authorization record should identify the domain or environment, permitted dates, accounts, test categories, rate limits, and emergency contact. The target allowlist is enforced at the network and browser layers, not merely described in a prompt. The default mode should be passive or low-impact; active checks require explicit policy enablement.

The tool layer should block destructive operations, credential guessing, privilege changes, mass account creation, data exfiltration, and actions that affect third parties. Authentication secrets must be injected through a secure test harness rather than placed in prompts or logs. Screenshots, traces, and video should be retained only as long as needed, with tokens and personal information redacted. Browser recordings are supporting context and should not be treated as independent proof.

LLM inputs may contain untrusted page text, including instructions embedded in a website. Such text must be treated as data, never as policy. The planner must not follow page content that attempts to change its system instructions, expand scope, reveal secrets, or request external actions. The tool adapter remains the enforcement boundary even if the model is manipulated.

Human review is required before a finding is marked as externally reportable or before a test with meaningful side effects is performed. The framework should support a “dry-run” mode that displays planned actions without executing them. A stop button and per-run cancellation mechanism should be available to the operator.

9 Prototype Workflow and Preliminary Observations

The current prototype context consists of a Node.js agent loop, a Playwright-controlled local web application, and a Groq-hosted GPT-OSS 20B model used for tool selection and interpretation. The observed log excerpt shows navigation among a local home route and an authentication route, repeated attempts to click a login button, and test strings entered into email-related fields. A separate summary reports observations of missing Content-Security-Policy, HSTS, X-Content-Type-Options, and framing protections on local routes. These are preliminary observations from a development run, not a controlled benchmark.

The logs also show symptoms that motivate the proposed architecture: repeated actions without demonstrated state progress, mixed or malformed output fragments, and natural-language report sections appearing in a location intended for findings. The repeated login clicks indicate that the agent needs post-action verification and a duplicate-action guard. The presence of script-like text in a field does not demonstrate XSS, because the logs do not show a subsequent execution context or a verified browser execution event. The header observations are useful hardening signals, but the available log excerpt does not establish that they are exploitable vulnerabilities or that they map to a specific CVE.

Accordingly, the prototype should be described as an implementation under development. The next engineering milestone is not to increase the number of tests, but to make the current observations deterministic and auditable: define schemas, validate each tool response, add state fingerprints, separate findings from narrative text, and require evidence before classification. After these controls are implemented, a labelled test suite can be used to measure detection and false-positive rates.

10 Evaluation Methodology

A defensible evaluation requires a controlled collection of applications and known outcomes. The dataset should contain intentionally vulnerable training applications, secure control applications, and a small set of organization-owned test applications where permission is documented. Each case should have a ground-truth record describing the vulnerability or hardening condition, the expected evidence, applicable CWE, and whether a CVE is relevant. Tests should be repeated across browser versions and model settings to quantify nondeterminism.

10.1 Research Questions

RQ1: Does state-aware loop handling reduce duplicate actions compared with a baseline agent loop? RQ2: Does structured tool validation reduce malformed outputs and orphaned findings? RQ3: Does evidence validation improve precision and reduce unsupported vulnerability claims? RQ4: How often can the system assign a justified CWE, and how often does it correctly abstain from CVE mapping when product/version evidence is absent? RQ5: What is the time and action cost of these controls?

10.2 Metrics

Finding precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is the harmonic mean of precision and recall. The false-positive rate is FP/(FP+TN), where TN and FP are computed over explicitly labelled test cases. Evidence completeness can be measured as the fraction of findings containing all required evidence fields. Loop rate can be measured as duplicate action-state pairs divided by total executed actions. Tool-schema validity is the fraction of tool responses that pass schema validation. CWE mapping accuracy is assessed against expert-reviewed labels; CVE mapping precision is measured only for cases with known affected products and versions. Runtime, number of browser actions, and human review time are secondary measures.

10.3 Baselines and Ablations

Compare (A) an unconstrained baseline loop, (B) a loop with step limits only, (C) step limits plus state-aware duplicate detection, and (D) the complete system with structured outputs and evidence validation. For the classifier, compare LLM-only interpretation against deterministic checks plus evidence validation. All configurations must use the same target set, browser setup, and permission scope. Report confidence intervals where the sample size supports them and publish the test definitions to support replication.

10.4 Reporting Integrity

No numerical detection result is reported in this paper because the necessary labelled evaluation has not been performed. The existing local logs are used only to motivate failure modes and design requirements. Any later results should include dataset composition, inclusion criteria, software versions, model identifier, prompt and tool-schema versions, test budgets, and adjudication rules. This avoids presenting anecdotal observations as generalizable performance claims.

11 Discussion

The main design principle is separation of concerns. The LLM can help navigate unfamiliar interfaces, but the browser adapter determines what actually happened. Deterministic checks can establish facts such as the absence of a header in a captured response; evidence validation determines whether those facts support a security claim; and taxonomy mapping adds standardized labels without changing the evidence. This structure makes failures easier to debug because a report can be traced to a tool call, a response, and a check.

Structured outputs also improve maintainability. If the tool response schema changes, the version can be incremented and validated. Report generation then operates over a stable internal representation rather than parsing arbitrary model prose. This is particularly important when a model returns a well-written summary that accidentally gets stored as a finding. Narrative quality is not evidence quality.

The framework has limitations. Browser-only visibility restricts what can be inferred about server-side code, access control, database queries, and infrastructure. Security header checks are contextual and cannot replace a threat model. State fingerprints can produce false duplicates if dynamic content changes or false differences if timestamps and random identifiers are not normalized. LLM planning remains nondeterministic, and strict budgets can reduce coverage. CVE matching is difficult when product identity and version cannot be reliably inferred. These limitations motivate human review and explicit abstention rather than stronger claims.

The proposed framework is also not a substitute for a mature security testing program. It is best positioned as a transparent assistant for bounded, repeatable browser checks, with outputs that can be consumed by engineers and reviewed by security specialists. Its value should ultimately be demonstrated through a reproducible evaluation rather than through a compelling demonstration alone.

12 Conclusion and Future Work

This paper presented a controlled framework for LLM-assisted browser security assessment using Playwright, structured tool outputs, evidence-based checks, and cautious CWE/CVE mapping. The design addresses repeated browser actions, malformed tool responses, unsupported vulnerability claims, and the conflation of generic weaknesses with known CVEs. It establishes explicit authorization scope, bounded execution, evidence provenance, confidence reporting, and human review as core requirements.

The current prototype logs provide useful development signals but do not support quantitative claims about detection accuracy. The immediate implementation priorities are a versioned tool schema, action budgets, state fingerprints, post-action verification, typed check outcomes, evidence references, and a report builder that cannot convert model prose directly into findings. The next research step is to evaluate these controls on a labelled set of authorized test applications, compare baseline and ablated configurations, and publish reproducible metrics.

Future work includes robust handling of single-page applications, redaction-aware trace storage, improved route prioritization, policy-driven authentication, expert-reviewed taxonomy mappings, and integration with authoritative vulnerability data. A successful system should be judged not by the number of findings it produces, but by whether each finding is accurate, explainable, reproducible, and appropriately scoped.

Acknowledgements

The authors acknowledge the use of open-source browser automation and security guidance resources that inform the proposed design. Any institutional, supervisory, or funding acknowledgement should be added only where applicable and approved by the relevant contributors.

References

[1] Microsoft: Playwright Documentation. https://playwright.dev/ (accessed 25 October 2026).

[2] OWASP Foundation: Web Security Testing Guide (WSTG). https://owasp.org/www-project-web-security-testing-guide/ (accessed 25 October 2026).

[3] MITRE: Common Weakness Enumeration (CWE). https://cwe.mitre.org/ (accessed 25 October 2026).

[4] CVE Program: Common Vulnerabilities and Exposures. https://www.cve.org/ (accessed 25 October 2026).

[5] National Institute of Standards and Technology: National Vulnerability Database (NVD). https://nvd.nist.gov/ (accessed 25 October 2026).

[6] FIRST: Common Vulnerability Scoring System (CVSS). https://www.first.org/cvss/ (accessed 25 October 2026).

[7] OWASP Foundation: OWASP Application Security Verification Standard (ASVS). https://owasp.org/www-project-application-security-verification-standard/ (accessed 25 October 2026).

[8] Springer Nature: Guidelines for Authors of Proceedings. Author-provided conference submission materials (accessed 25 October 2026).

Appendix A: Recommended Finding Schema

A minimal finding record should include: finding_id; title; status; check_id; target_url; observed_at; observation; expected_condition; evidence[]; confidence; severity; severity_rationale; cwe_candidates[]; cve_match; remediation; limitations; reviewer_status. The cve_match object should be null unless a supported match exists and should otherwise include the identifier, affected product, version evidence, advisory source, and applicability rationale. The evidence array should refer to stored artifacts by stable IDs rather than embedding secrets or large response bodies.

Appendix B: Release Checklist

Before each release, verify that (1) the target allowlist is enforced; (2) actions and navigation have budgets; (3) duplicate action-state pairs are blocked; (4) tool arguments and outputs are schema-validated; (5) timeouts distinguish failed from uncertain actions; (6) LLM prose cannot be parsed as a finding without validation; (7) header checks preserve raw response evidence; (8) XSS tests require a verified execution condition in an authorized fixture; (9) CWE mapping includes rationale; (10) CVE mapping requires product/version applicability evidence; (11) screenshots, traces, and videos are redacted; (12) findings can be reproduced; and (13) evaluation results are not reported until a labelled test set has been run.